AgPlane: Programmable OS-Level IFC Policy Enforcement for AI Agent Harnesses

Paper #479
Anonymous Submission

ACM Reference Format:
Paper #479. 2026. AgPlane: Programmable OS-Level IFC Policy Enforcement for AI Agent Harnesses. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26).* ACM, New York, NY, USA, 4 pages.

AI agents are widely used for coding, DevOps, and enterprise workflows[9, 26, 28–30], run on real infrastructure and handle sensitive data. Besides the LLM, agents require AI agent harnesses: software around the model that improves agent performance while enforcing instructions and constraints for safety and compliance [2].

A major component of the harness is a policy engine [23]: the part that observes and enforces instructions and constraints over the agent's concrete actions. Projects encode many such policies in instruction files (e.g. `CLAUDE.md`, `AGENTS.md`) [4, 5, 17, 20], so harnesses can provide them to the model directly. Because LLMs comply with instructions probabilistically [15, 16, 18] and may violate them through planning errors, prompt-injection drift, and opaque tool or script behavior [9, 14, 29], harnesses also require deterministic policy engines to enforce compliance [8, 19, 25].

However, existing policy engines leave a *semantic gap*: *policy intent* is expressed in underspecified natural language, but enforcement needs to decide over concrete *system actions*. Our empirical study of instruction-files in 64 popular projects finds that 64% of statements are behavioral directives, and 83% of those involve system-level behavior: commands executed, files accessed, and network connections made. Existing tool-call-based policy systems [7, 8, 21, 25, 27] in agent runtimes such as Claude Code [1], whether LLM guardrails or regex-like checkers, lose track of indirect system actions when agents spawn subprocesses: e.g. the same `git push` can be in a `bash` wrapper created in a previous session; and at the repository level, 81% of projects contain cross-event system directives that are hard to enforce with stateless single-call interception (e.g., "run tests before commit").

OS-level enforcement with static policy is also insufficient. Most rules reference project or task context that requires resolving , (e.g., "never modify upstream source," where *upstream source* needs to be resolved from the repository; "do not update dependencies unless requested"; "the reviewer sub-agent only writes to its project directory"), making it hard for human administrators to pre-define policy for them without context. We find that 74% of system-level directives require intent-level context outside of low-level OS events. OS-level enforcement systems and sandboxes for traditional software [3, 6, 11, 13, 22] can observe system actions but expect pre-defined static policy and return only system events without semantic context, leaving the agent unlikely to follow its policies.

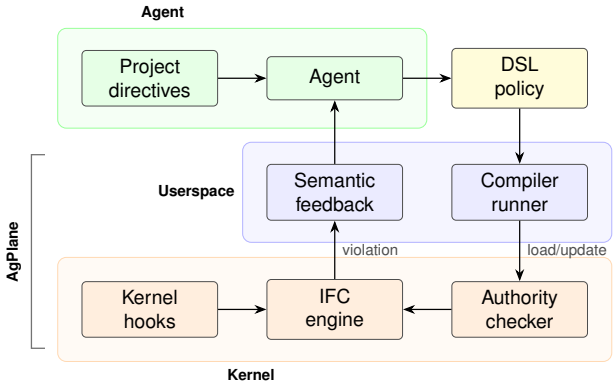

Figure 1. AgPlane architecture: policy generation, compilation, kernel enforcement, and feedback loop.

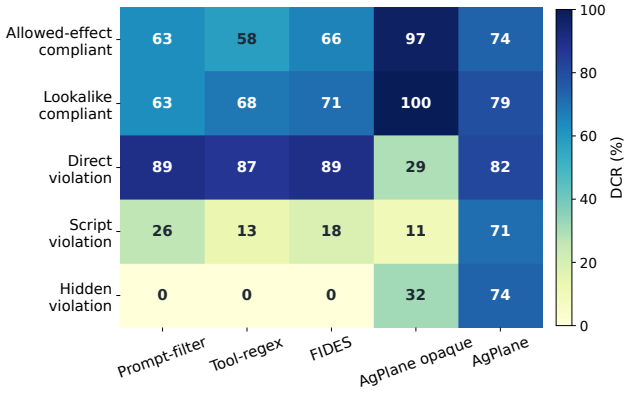

Figure 2. Decision Compliance Rate (DCR, %) by execution-path type and system. DCR is the fraction of traces where the system produces the correct enforcement outcome. Rows are path types: *allowed-effect/lookalike* are compliant traces; *direct* violations use a tool call; *script* violations split the action across a subprocess; *hidden* violations embed the action in an artifact whose entrypoint appears benign. AgPlane's advantage concentrates on script and hidden paths invisible to tool-call baselines.

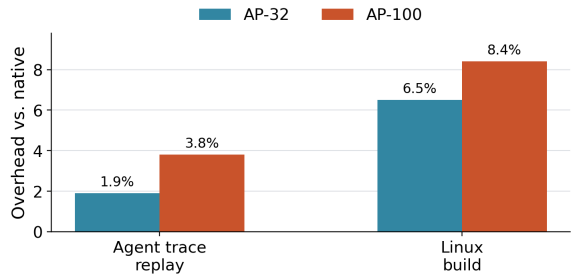

Figure 3. End-to-end overhead normalized to native execution, for agent trace replay and Linux kernel build.

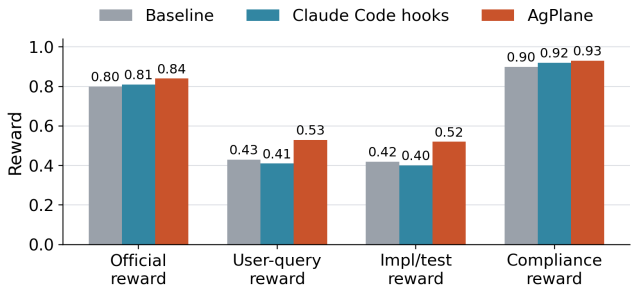

Figure 4. Coding-task reward on a 21-task subset of OctoBench [10], broken down by system.

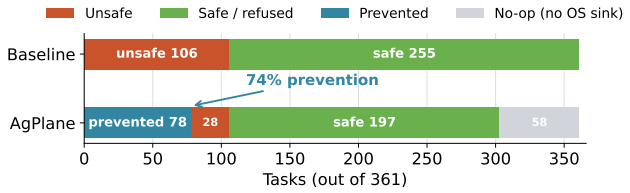

Figure 5. Outcome distribution on 361 OpenAgentSafety [24] tasks. AgPlane reduces baseline-unsafe outcomes from 106 to 28 (74% prevention rate).

To bridge the gap, we argue that an agent-harness policy engine should be programmable by agents and take effect at OS level. It should let agents use live project and task context to define policy and enforce actions. While safety constraints come from higher authority, we observe that the required context is mainly within agents closest to the task: they already read the repository, interpret the current task, and resolve abstract references into commands and paths. This makes agents the natural producer of concrete compliance policy, also reflecting the fact that instruction files are increasingly maintained by agents [1, 12].

To realize this, first, policy specification needs to be high-level enough for agents to easily generate from natural language with minimal token consumption, yet concrete enough to compile to deterministic kernel checks. Second, enforcement needs to track policy across the system without imposing prohibitive overhead. Third, since many rules are about safety, programmability must not become an authority bypass: agents cannot weaken inherited constraints set by separate trusted administrators, safety rules should be installed separately.

AgPlane is a runtime-agnostic, programmable OS-level policy enforcement system for AI agent harnesses. Agents express policies in a DSL; AgPlane compiles them into labeled information-flow rules and loads them into an eBPF engine with processes, files, and network kernel hooks. Each rule has one of three effects: notify (observe) for instructions, block (pre-operation denial on hooks) or kill (process termination) for constraints, all with semantic feedback about why and what to do next. A layered authority model ensures that higher-authority rules cannot be weakened by lower-authority agents. On decision-compliance benchmark, AgPlane resolves 2.0–3.2× more violations than prompt-filter, tool-regex, FIDES [7] (tool-level IFC), and feedback-free kernel IFC by covering indirect execution paths that tool-call interception cannot, while adding 1.9% end-to-end overhead on agent workloads and up to 8.4% on kernel builds. On 21 real coding tasks (Figure 4), a policy agent submitted 110 runtime deltas to refine rules for the task-executing sub-agent, improving task reward over baseline and tool-call hooks. On safety benchmark of 361 personal-assistant tasks, AgPlane prevents 74% of baseline-unsafe behaviors using separate agent-generated policies before unsafe execution as in higher authority path.

To summarize, we make three contributions:

1. An **empirical study** of 64 projects that characterizes these gaps and motivates AgPlane.
2. **AgPlane**, a programmable OS-level policy enforcement system that addresses them.
3. An **evaluation** on our decision-compliance benchmark building on empirical study, external coding-task and safety benchmarks covering workplace and personal-assistant tasks.

References

[1] Anthropic. 2025. Claude Code. https://code.claude.com/docs.
[2] Birgitta Böckeler. 2026. Harness Engineering for Coding Agent Users. https://martinfowler.com/articles/harness-engineering.html. Published April 2, 2026.
[3] Canonical Ltd. 2024. AppArmor Security Profiles. https://apparmor.net/.
[4] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. https://arxiv.org/abs/2511.12884
[5] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2026. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. In *Product-Focused*

Software Process Improvement. Springer Nature Switzerland, 543–551. doi:10.1007/978-3-032-12089-2_40

[6] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io/.

[7] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. https://arxiv.org/abs/2505.23643

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 82895–82920. doi:10.52202/079017-2636

[10] Deming Ding, Shichun Liu, Enhui Yang, Jiahang Lin, Ziying Chen, Shihan Dou, Honglin Guo, Weiyu Cheng, Pengyu Zhao, Chengjun Xiao, Qunhong Zeng, Qi Zhang, Xuanjing Huang, Qidi Xu, and Tao Gui. 2026. OctoBench: Benchmarking Scaffold-Aware Instruction Following in Repository-Grounded Agentic Coding. arXiv:2601.10343. https://arxiv.org/abs/2601.10343

[11] Jake Edge. 2015. A seccomp overview. https://lwn.net/Articles/656307/.

[12] Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. 2026. Configuring Agentic AI Coding Tools: An Exploratory Study. In *ACM AIware 2026*. OpenReview.net. https://openreview.net/forum?id=cqmx1MLZCq

[13] Google. 2018. gVisor: Application Kernel for Containers. https://github.com/google/gvisor. https://gvisor.dev/

[14] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, Copenhagen, Denmark, 79–90. doi:10.1145/3605764.3623985

[15] Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. 2024. FollowBench: A Multi-level Fine-grained Constraints Following Benchmark for Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 4667–4688. doi:10.18653/v1/2024.acl-long.257

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[17] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. In *Proceedings of the 1st Journal Ahead Workshop at the International Conference on Software Engineering*. Association for Computing Machinery. https://conf.researchr.org/details/icse-2026/jaws-2026-papers/31/On-the-Impact-of-AGENTS-md-Files-on-the-Efficiency-of-AI-Coding-Agents JAWs@ICSE 2026.

[18] Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. 2025. AGENTIF: Benchmarking Large Language Models Instruction Following Ability in Agentic Scenarios. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2025/hash/51bb3a8a33610a25aae074bfc51b1b1f-Abstract-Datasets_and_Benchmarks_Track.html

[19] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Singapore, 431–445. doi:10.18653/v1/2023.emnlp-demo.40

[20] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2026. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. In *Proceedings of the 2026 International Workshop on Agentic Engineering*. Association for Computing Machinery, Rio de Janeiro, Brazil, 63–67. doi:10.1145/3786167.3788412

[21] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

[22] The Linux Kernel Documentation. 2025. Landlock: Unprivileged Access Control. https://www.kernel.org/doc/html/latest/userspace-api/landlock.html.

[23] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. https://www.langchain.com/blog/the-anatomy-of-an-agent-harness. Published March 10, 2026.

[24] Sanidhya Vijayvargiya, Aditya Bharat Soni, Xuhui Zhou, Zora Zhiruo Wang, Nouha Dziri, Graham Neubig, and Maarten Sap. 2026. OpenAgentSafety: A Comprehensive Framework for Evaluating Real-World AI Agent Safety. In *The Fourteenth International Conference on Learning Representations*. OpenReview.net. https://openreview.net/forum?id=xggSxCFQbA

[25] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 12 pages. https://conf.researchr.org/track/icse-2026/icse-2026-research-track Research Track; preprint available at https://arxiv.org/abs/2503.18666.

[26] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. OpenReview.net, Singapore, 38 pages. https://openreview.net/forum?id=OJd3ayDDoF

[27] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Vancouver, Canada, 68316–68342. https://openreview.net/forum?id=2nBcjCZrrP

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 50528–50652. doi:10.52202/079017-1601

[29] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-

acl.624

[30] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*. Association for Computing Machinery, Seoul, Republic of Korea, 110–115. doi:10.1145/3766882.3767169